

um-game-playing-strength-of-mcts-variants

2nd place solution

Rank	2
Team	Richard_U
Author	Richard_U
Topic ID	549718
Version	Original

Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549718>

Retrieved with Kaggle CLI on 2026-07-03.

First, I want to thank the competition host Maastricht University and Kaggle for hosting this competition.

I also want to thank the Kaggle community for giving discussions and generous sharing of ideas. Especially I want to thank [yunsuxiaozi](#) for the [MCTS Starter notebook](#), wherefrom I adopted the idea of calculating ARI, McAlpine EFLAW and CLRI scores from the 'LudRules' column. Thank you!

As many others, I was expecting a big shake-up and I really did not anticipate my solution to be among the top ones. Therefore, I was this morning surprised to discover that the top three positions were unchanged between the public and private leaderboard.

Update:

Link to submission notebook: [2nd place solution - submission](#)

Link to model dataset, with instructions on how to reproduce the model binaries (offline training): [um-gpsomctsv-2nd-place-solution](#)

Overview

In summary, I think my solution can be described as embarrassingly straight forward:

- Traditional 5-split Cross Validation (CV) training
- No additional offline generated training data
- No fancy feature engineering (aside from the mentioned text-scores from LudRules)
- No feature selection (aside from dropping non-information/duplicate features and dropping both the rule-set features)

The big breakthrough in performance clearly came through data augmentation. I used exactly the same data augmentation as the [3rd place solution](#). I think this was a pretty natural approach given the training data and I'm sure many others, independently from each other, used exactly this augmentation.

In contrast to what others have reported (for example [3rd place solution](#)), I had absolutely no luck experimenting with out of fold (OOF) predictions as a feature in a second training run. (However, I did not spend much time on this and maybe I just did some kind of typo messing things up!) Therefore my models simply used a regular one-run training/prediction structure.

My final overall model consisted of a weighted ensemble of 7 models, whereof 4 boosting and 3 Neural Network (NN) models. Finally the overall submission used an equally weighted mix of 6 sets of identically structured model sets, with the used seed for training as only difference.

Data preparation and CV split selection

- Duplicate and non-information features were dropped.
- ARI, McAlpine EFLAW and CLRI scores were calculated from the LudRules column.
- Data was sorted on groups created from the 'LudRules' column. Groups were selected through Tfidf Vectorization and Kmeans clustering with a slight reduction from the initial 1373 rule-sets down to approximately 1270 (slightly different number for different seeds). The intention was to treat very similar

games as just one game when creating CV splits and thereby generate CV estimates that hopefully generalized better outside the training data. I'm not sure if this actually helped though, but it seemed to make sense and at least to not have a negative impact on prediction performance.

- 'LudRules' 'EnglishRules', 'GameRulesetName' and 'Id' were dropped.
- Data was augmented through flipping 'agent1' and 'agent2', change the 'AdvantageP1' to 1-'AdvantageP1' and sign flip change 'utility_agent1' to 'utility_agent1'*-1. I also added an augmented data dummy feature to let the models know which rows of the data that originated from augmentation. This might seem counter-intuitive but it improved CV scores slightly. The augmentation was also used in inference, thus generating two predictions from each model – with the augmented one flipped back by multiplication of -1.
- The categorical agent columns values were one-hot dummy encoded (with the most common value used as baseline).
- Unique rule-set groups from 'LudRules' were dummy-encoded. These additional rule group features were only used when training some of the boosting models. (The addition of these “all-zero-outside-training”-features did not seem to be beneficial for, and was therefore not used in, the NN models.)
- The continuous variables were mapped to standard normal distributions before training of, and inference from, the NN models.

Other than the text scores from the 'LudRules' column, thanks again to [yunsuxiaozi](#), I did not copy any ideas from the public notebooks. Still I most surely benefited from the interesting discussions and Kaggle community overall!

Model structure and training

Ensemble of 7 models

1. **Boosting:** CatBoost without dummy rule-set identifiers
2. **Boosting:** 2nd CatBoost with dummy rule-set identifiers
3. **Boosting:** LightGBM (LGBM) without dummy rule-set identifiers
4. **Boosting:** 2nd LGBM with dummy rule-set identifiers
5. **NN:** MultiLayer Perceptron (MLP) without dummy rule-set identifiers
6. **NN:** 2nd, just bigger, MLP without dummy rule-set identifiers
7. **NN:** Auto-Encoder (AE) followed by a MLP without dummy rule-set identifiers

All models used the same features, except for the addition of the full set of dummy rule-group-set identifier features to boosting model 2 and 4.

Predictions from each model were capped to the target range (-1 to 1). The models were then combined to a CV weighted ensemble, with weights using an Ordinary Least Squares (OLS) regression. Augmented predictions from the NN models were excluded at this stage, this as they seemed to provide no clear benefit. Also, somewhat unexpectedly, the CV estimates gave clear negative weights for the predictions from both the 2nd bigger MLP NN (model 6) and the AE NN (model 7). Generally I'm skeptical to use negative model weights where positive is expected. I consider negative estimates to primarily be an indication of improvement potential in the model structure/specification, and not something to actually include in a model ensemble.

However, I did not manage to improve my approach within the competition time limit and, as the negative weights seemed beneficial for both the CV- and public leaderbord-score, I decided to go for the full model structure including the two negative weighted NN models. Also, an hypothesis that negative weights – for possibly overfitting models – could somewhat improve generalized performance, i.e. outside the training data, did not appear as totally unreasonable to me.

The full ensemble predictions were then capped to the target range and the capped ensemble predictions were scaled through CV OLS regressions. I was somewhat afraid that this could overfit the data, but I still decided to keep the scaling as it improved both the CV and public leaderboard score.

Using the above structure, I trained 6 identically structured model sets with different seeds. The seeds were semi-randomized, with some manual handpicking where CV scores too low or too high were discarded. This as more moderate CV scores seemed to generalize better to the public leaderboard.

Finally in an absolutely last step, the combined predictions from all 6 sets were again capped to the target range.

Final adjustments through direct public leaderboard (over-)fitting

With the approach described in the previous section I generated a mostly CV based submission.

In addition I decided to do another one based on what I considered to be more of a full leaderboard overfit. The possibly “overfit”-approach differed from the first in two ways:

1. The previously excluded augmented data predictions from the NN models were now included, with negative weights fitted directly to the public leaderboard.
2. A prediction set with equal weights to all model predictions were added. These predictions were then added with, again negative, weights fitted directly from the public leaderboard.

To me this seemed to be a classic overfit and almost desperate move, almost certain not to be a winning choice for the private leaderboard score.

However, the scores for my CV based versus public leaderboard overfit strategies turned out as follows:

	Private score	Public score
Mostly CV based submission	0.42324	0.41779
Full public leaderboard overfit submission	0.41996	0.41429

So, luckily for me, I was wrong. The dubious choice of a leaderboard overfit approach turned out to be rewarding in this particular competition. At least in part this could be explained by that some game rulesets appeared in both the public and the private test set, see this [End of Competition post](#) by the organizers. I did not expect games to be present in both public and private test sets, but it's quite clear that it rewards public leaderboard aligned submissions. In any case, I'm humbled by the result and honestly consider my 2nd place submission to be more luck than skill.

Again, thanks